

2.6 Launch 文件与参数系统

所属阶段： 第二阶段 · ROS 2 核心通信机制

讲次数量： 12 节

预计时长： ~70 分钟

前置依赖： 2.5 自定义接口 (Interfaces)

学习目标

完成本节后，学员将能够：

- 解释 Launch 文件在 ROS 2 系统启动中的角色，区分 XML 与 Python 两种写法
- 编写 XML 和 Python Launch 文件，启动单节点与多节点系统
- 在 Launch 文件中配置重映射 (Remapping) 与命名空间 (Namespace)
- 理解 ROS 2 Parameter 的概念、作用域与生命周期
- 在 Python (rclpy) 和 C++ (rclcpp) 节点中声明、读取和动态更新参数
- 使用 YAML 文件集中管理参数，并在 Launch 文件中加载
- 实现参数回调 (Parameter Callbacks)，在运行时校验和响应参数变更
- 通过 Turtlesim 实验掌握 `ros2 param` 命令行工具
- 编写综合 Launch 文件，一键启动 turtlesim + teleop + 自定义节点

讲次 1：什么是 Launch 文件

知识点

在 2.1 ~ 2.5 中，我们多次使用 `ros2 run` 在多个终端分别启动节点。当系统只有两三个节点时尚可接受，但随着节点数量增加，手动管理变得低效且容易出错：

- 需要记住每个节点的 Package 名和可执行文件名
- 需要为每个节点单独配置重映射、命名空间和参数
- 节点启动顺序、依赖关系难以保证
- 团队协作时，新人难以复现相同的运行环境

Launch 文件正是为解决这些问题而设计的——它是一个**声明式的系统启动脚本**，用一份文件描述"要启动哪些节点、如何配置它们"，然后用一条命令全部拉起。

Launch 文件的核心价值

能力	说明
一键启动	单条命令启动整个机器人系统（感知 + 规划 + 控制）
配置集中	重映射、命名空间、参数统一写在 Launch 文件中
可复现	团队成员使用同一份 Launch 文件，环境完全一致
可版本管理	Launch 文件纳入 Git，随代码一起迭代
条件逻辑	Python Launch 支持 if/else、循环等动态逻辑（XML 不支持）

两种 Launch 文件格式

ROS 2 支持两种 Launch 文件格式：

格式	扩展名	特点
XML	<code>.launch.xml</code> 或 <code>.xml</code>	结构直观，适合简单静态配置
Python	<code>.launch.py</code>	功能强大，支持动态逻辑和复杂编排

两种格式最终都通过 `ros2 launch` 命令执行，底层由 `launch` 和 `launch_ros` 包解析。

基本使用流程

开发阶段

编写 Launch 文件



安装到 `share/<pkg>/launch/`



`colcon build`

运行阶段

`ros2 launch <pkg> <file>`



解析并启动所有节点



系统就绪

第一个体验：直接启动官方 Launch

无需编写代码，先用系统自带的 Launch 文件感受效果：

```
source /opt/ros/jazzy/setup.bash
ros2 launch turtlesim multisim.launch.py
```

该命令会同时启动两只海龟。打开新终端观察：

```
ros2 node list
# 预期输出类似：
# /turtlesim
# /turtlesim2
```

关闭所有节点只需在 Launch 终端按 `Ctrl+C` ——Launch 系统会负责优雅地关闭所有子进程。

Launch 文件在 Package 中的位置

Launch 文件通常放在 Package 的 `launch/` 目录下，构建后安装到 `install/<pkg>/share/<pkg>/launch/`：

```
my_robot_bringup/      ← 常用于存放 Launch 的"启动"Package（示意结构）
├─ launch/
│  └─ turtlesim.launch.xml
│  └─ bringup.launch.py
├─ config/
│  └─ bringup.yaml
├─ CMakeLists.txt      ← ament_cmake: install launch 目录
└─ package.xml
```

Launch 与 `ros2 run` 的对比

维度	<code>ros2 run pkg node</code>	<code>ros2 launch pkg file</code>
启动节点数	单个	一个或多个
配置方式	命令行 <code>--ros-args</code>	Launch 文件内声明
适用场景	快速调试单个节点	正式启动整个系统
可维护性	低（命令难记）	高（文件可版本管理）

关键点

- Launch 文件是 ROS 2 系统启动的声明式脚本，解决多节点管理的痛点
 - 支持 XML（简单静态）和 Python（动态逻辑）两种格式
 - 使用 `ros2 launch <pkg> <file>` 执行，一条命令启动整个系统
 - Launch 文件通常放在 `launch/` 目录，随 Package 一起安装
 - 日常开发调试可用 `ros2 run`，系统集成和部署推荐 Launch 文件
-

讲次 2：创建 XML Launch 文件

知识点

XML Launch 是 ROS 2 最直观的启动配置方式，语法类似 HTML，适合“节点列表固定、配置不变”的场景。本节在 `my_robot_bringup` Package 中创建第一个 XML Launch 文件。

创建 bringup Package

```
cd ~/ros2_ws/src
ros2 pkg create my_robot_bringup --build-type ament_cmake --dependencies rclcpp
mkdir -p my_robot_bringup/launch
```

最小 XML Launch 文件

创建 `my_robot_bringup/launch/turtlesim.launch.xml`：

```
<launch>
  <node pkg="turtlesim" exec="turtlesim_node" name="turtlesim"/>
</launch>
```

三个核心属性：

属性	含义	示例
<code>pkg</code>	节点所在的 ROS 2 Package	<code>turtlesim</code>
<code>exec</code>	可执行文件名（不含路径）	<code>turtlesim_node</code>
<code>name</code>	运行时节点名称（可覆盖默认名）	<code>turtlesim</code>

注意：ROS 2 使用 `exec` 而非 ROS 1 的 `type`，这是 ROS 2 XML Launch 与 ROS 1 的关键区别。

安装 Launch 文件

在 `CMakeLists.txt` 中添加：

```
install(DIRECTORY launch
  DESTINATION share/${PROJECT_NAME}
)
```

构建并运行：

```
cd ~/ros2_ws
colcon build --packages-select my_robot_bringup
source install/setup.bash
ros2 launch my_robot_bringup turtlesim.launch.xml
```

启动多个节点

扩展 Launch 文件，同时启动仿真器和键盘控制：

```
<launch>
  <node pkg="turtlesim" exec="turtlesim_node" name="turtlesim"/>
  <node pkg="turtlesim" exec="turtle_teleop_key" name="teleop"
    output="screen"
    emulate_tty="true"
    launch-prefix="bash -lc 'exec &lt;/dev/tty; exec &quot;${@}&quot;' bash"/>
</launch>
```

保存为 `turtlesim_teleop.launch.xml`，一条命令即可操作海龟：

```
ros2 launch my_robot_bringup turtlesim_teleop.launch.xml
```

若在 WSL、IDE 集成终端或远程伪终端中运行，`turtle_teleop_key` 需要拿到真实键盘输入。上面的 `emulate_tty` 与 `launch-prefix` 用于把键盘节点重新接回当前终端；普通 Linux 桌面终端中也可以只保留 `output="screen"`。

常用 XML 子元素

```
<launch>
  <!-- 启动节点 -->
  <node pkg="turtlesim" exec="turtlesim_node" name="turtlesim"
    namespace="robot1"
    output="screen">
    <!-- 话题重映射（使用相对名称，配合 namespace 自动加前缀） -->
    <remap from="turtle1/cmd_vel" to="cmd_vel"/>
    <!-- 内联参数 -->
    <param name="background_r" value="0"/>
    <!-- 从 YAML 加载参数（讲次 10 详述） -->
    <param from="$(find-pkg-share my_robot_bringup)/config/turtlesim.yaml"/>
  </node>
</launch>
```

子元素 / 属性	作用
<code><remap from="..." to="..." /></code>	话题/服务重映射
<code><param name="..." value="..." /></code>	设置单个参数
<code><param from="..." /></code>	从 YAML 文件加载参数
<code>namespace="..."</code>	设置节点命名空间
<code>output="screen"</code>	将节点日志输出到终端

`$(find-pkg-share)` 替换符

`$(find-pkg-share <pkg>)` 是 Launch 文件的**替换 (Substitution)** 机制，在运行时被解析为 Package 的 share 目录绝对路径：

```
$(find-pkg-share my_robot_bringup)/config/turtlesim.yaml
↓ 运行时解析为
/home/user/ros2_ws/install/my_robot_bringup/share/my_robot_bringup/config/turtlesim.yaml
```

这确保了无论工作空间安装在哪里，路径始终正确。

关键点

- XML Launch 根元素为 `<launch>`，每个节点用 `<node pkg="..." exec="..." name="..." />` 声明
 - ROS 2 使用 `exec` 属性（非 ROS 1 的 `type`）
 - 通过 `install(DIRECTORY launch ...)` 将 Launch 文件安装到 share 目录
 - `ros2 launch <pkg> <file>` 执行，支持 `.launch.xml` 和 `.xml` 扩展名
 - `$(find-pkg-share pkg)` 用于引用 Package 内的资源文件路径
-

讲次 3：创建 Python Launch 文件（与 XML 对比）

知识点

Python Launch 文件功能远超 XML——可以使用变量、条件判断、循环，以及 Launch 系统提供的各种 Action 和 Substitution。对于复杂机器人系统，Python Launch 是社区首选。

最小 Python Launch 文件

创建 `my_robot_bringup/launch/turtlesim.launch.py`：

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='turtlesim',
            executable='turtlesim_node',
            name='turtlesim',
        ),
    ])

```

关键结构说明

元素	作用
<code>generate_launch_description()</code>	必须的入口函数，Launch 系统调用它获取启动描述
<code>LaunchDescription([...])</code>	启动描述容器，包含所有要执行的 Action
<code>Node(...)</code>	启动一个 ROS 2 节点的 Action
<code>package</code>	对应 XML 的 <code>pkg</code>
<code>executable</code>	对应 XML 的 <code>exec</code>

多节点启动

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    turtlesim_node = Node(
        package='turtlesim',
        executable='turtlesim_node',
        name='turtlesim',
    )

    teleop_node = Node(
        package='turtlesim',
        executable='turtle_teleop_key',
        name='teleop',
        output='screen',
        emulate_tty=True,
        prefix='bash -lc \'exec </dev/tty; exec "$@"\' bash',
    )

    return LaunchDescription([
        turtlesim_node,
        teleop_node,
    ])
```

构建并运行：

```
colcon build --packages-select my_robot_bringup
source install/setup.bash
ros2 launch my_robot_bringup turtlesim.launch.py
```

XML vs Python 对比

维度	XML Launch	Python Launch
语法	声明式标签	Python 代码
动态逻辑	不支持	支持 if/else、循环、函数
可读性	简单场景更直观	复杂场景更清晰
调试	较难	可用 Python 调试器
社区趋势	简单项目、遗留项目	主流选择
参数传递	有限	灵活（Launch Arguments）

Python Launch 的独有优势：Launch Arguments

Python Launch 支持在命令行传入参数，实现同一份 Launch 文件适配不同场景：

```

from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    # 声明一个可在命令行覆盖的参数
    use_sim_time_arg = DeclareLaunchArgument(
        'use_sim_time',
        default_value='false',
        description='Use simulation clock if true',
    )

    turtlesim_node = Node(
        package='turtlesim',
        executable='turtlesim_node',
        name='turtlesim',
        parameters=[{
            'use_sim_time': LaunchConfiguration('use_sim_time'),
        }],
    )

    return LaunchDescription([
        use_sim_time_arg,
        turtlesim_node,
    ])

```

运行时覆盖默认值（把上面内容保存为 `launch/sim_time_demo.launch.py` 后执行）：

```
ros2 launch my_robot_bringup sim_time_demo.launch.py use_sim_time:=true
```

注意：只能覆盖 Launch 文件里**已声明**的 Argument。仓库的 `turtlesim.launch.py` 声明的是 `bg_r`（见后文），对它传 `use_sim_time` 会直接报错。

引用 Package 内资源文件

Python Launch 中获取 share 目录路径：

```

from ament_index_python.packages import get_package_share_directory
import os

def generate_launch_description():
    pkg_share = get_package_share_directory('my_robot_bringup')
    config_file = os.path.join(pkg_share, 'config', 'turtlesim.yaml')

    return LaunchDescription([
        Node(
            package='turtlesim',
            executable='turtlesim_node',
            name='turtlesim',
            parameters=[config_file],
        ),
    ])

```

这等价于 XML 中的 `$(find-pkg-share my_robot_bringup)/config/turtlesim.yaml`。

何时选择哪种格式

节点数量 ≤ 3 ，配置固定，无动态逻辑？

- └ 是 → XML Launch（够用且简洁）
- └ 否 → Python Launch（推荐）

关键点

- Python Launch 入口函数为 `generate_launch_description()`，返回 `LaunchDescription([...])`
 - `Node(package=..., executable=..., name=...)` 对应 XML 的 `<node pkg exec name>`
 - Python Launch 支持 Launch Arguments、条件逻辑等高级特性，是复杂项目的首选
 - `get_package_share_directory()` 用于获取 Package 资源路径
 - 两种格式可共存于同一 Package，按场景选择
-

讲次 4: Launch 文件中的重映射与命名空间 (Remappings / Namespaces)

知识点

在 2.2 中，我们学习了用 `--ros-args -r` 在命令行做话题重映射。Launch 文件提供了更优雅、可版本管理的方式来配置重映射和命名空间。

为什么需要重映射和命名空间

考虑一个双机器人系统，每台机器人各有一个激光雷达 ROS 2 驱动节点（用户态程序，通过厂商 SDK 与硬件交互，非 Linux 内核驱动）和导航节点：

问题：两个 `lidar_node` 都发布 `/scan` → 话题名冲突！

解决：命名空间 `/robot1/scan` 和 `/robot2/scan`

问题：导航节点订阅 `/scan`，但驱动发布 `/lidar/scan` → 话题名不匹配！

解决：重映射 `/scan` → `/lidar/scan`

XML Launch 中的重映射

```
<launch>
  <!-- 使用相对名称重映射（自动添加命名空间前缀） -->
  <node pkg="turtlesim" exec="turtlesim_node" name="turtlesim"
        namespace="robot1">
    <remap from="turtle1/cmd_vel" to="cmd_vel"/>
  </node>
  <!-- 效果：/robot1/turtle1/cmd_vel → /robot1/cmd_vel -->
</launch>
```

重要：使用 namespace 时，remap 中应使用**相对名称**（不以 `/` 开头），这样命名空间会自动作为前缀。如果使用绝对名称（如 `/turtle1/cmd_vel`），命名空间不参与匹配，可能导致重映射无效。

Python Launch 中的重映射

```
Node(
    package='turtlesim',
    executable='turtlesim_node',
    name='turtlesim',
    namespace='robot1',
    remappings=[('turtle1/cmd_vel', 'cmd_vel')], # 相对名称，配合 namespace
)
```

规范写法：`remappings=[("/chatter", "/my_topic")]` ——元组列表，第一个元素是节点内部名称，第二个是系统中的实际名称。

命名空间 (Namespace)

命名空间为节点及其所有接口（话题、服务、参数）添加统一前缀：

```
节点名:    /robot1/turtlesim
话题:      /robot1/turtle1/pose
服务:      /robot1/spawn
参数:      /robot1/turtlesim.background_r
```

XML 写法：

```
<node pkg="turtlesim" exec="turtlesim_node" name="turtlesim" namespace="robot1"/>
```

Python 写法：

```
Node(
    package='turtlesim',
    executable='turtlesim_node',
    name='turtlesim',
    namespace='robot1',
)
```

双机器人完整示例

dual_robots.launch.py :

```

from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    robot1 = Node(
        package='turtlesim',
        executable='turtlesim_node',
        name='turtlesim',
        namespace='robot1',
    )

    robot2 = Node(
        package='turtlesim',
        executable='turtlesim_node',
        name='turtlesim',
        namespace='robot2',
    )

    teleop1 = Node(
        package='turtlesim',
        executable='turtle_teleop_key',
        name='teleop',
        namespace='robot1',
        output='screen',
        emulate_tty=True,
        prefix='bash -lc \'exec </dev/tty; exec "$@"\' bash',
        # 无需额外 remap: teleop 在 robot1 命名空间内,
        # 其 turtle1/cmd_vel 自动变为 /robot1/turtle1/cmd_vel,
        # 与 robot1 的 turtlesim_node 话题一致
    )

    return LaunchDescription([robot1, robot2, teleop1])

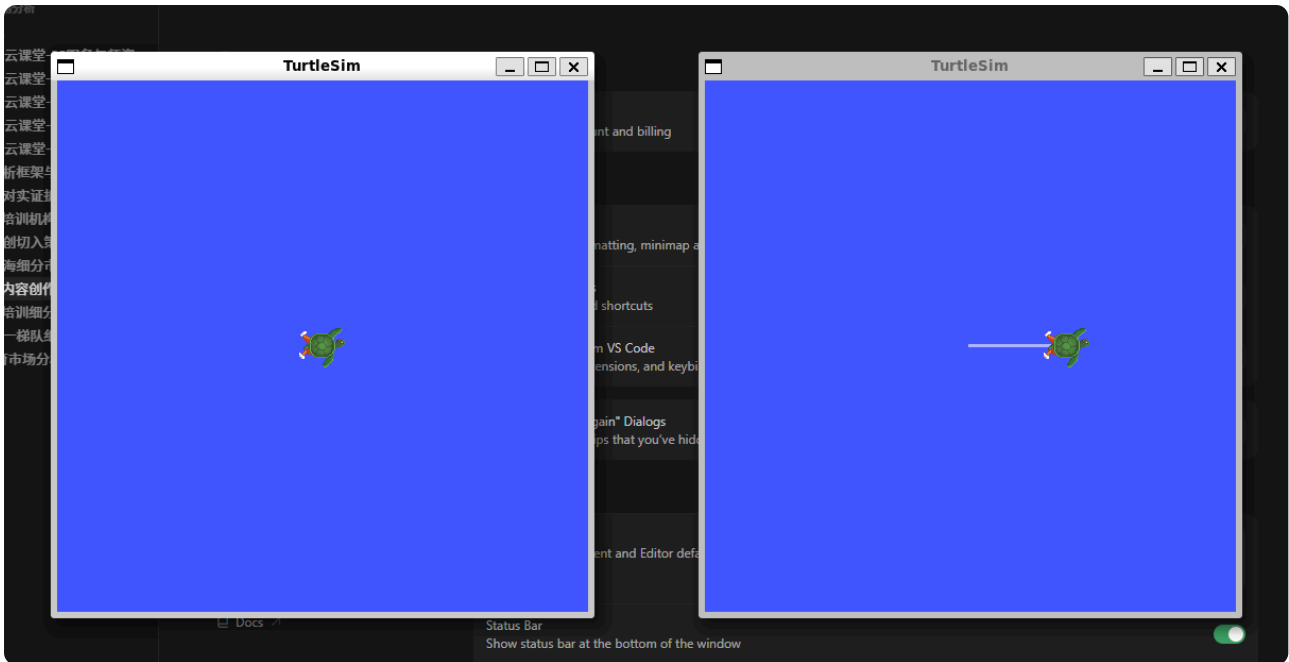
```

验证:

```

ros2 launch my_robot_bringup dual_robots.launch.py
ros2 node list
# /robot1/turtlesim
# /robot1/teleop
# /robot2/turtlesim

```



重映射的几种写法对比

写法	说明
<code>("chatter", "my_topic")</code>	相对名称，自动拼接命名空间
<code>("/chatter", "/my_topic")</code>	绝对名称，不受命名空间影响
<code>("turtle1/cmd_vel", "cmd_vel")</code>	节点内相对名 → 全局绝对名

最佳实践： 在团队内保持一致的命名约定，namespace 配合相对名称或统一使用绝对路径均可。

重映射 vs 改源码

方式	优点	缺点
改源码中的话题名	直观	不同部署场景需维护多份代码
Launch 重映射	一份代码适配多种部署	需理解重映射规则

社区共识：**源码中使用通用默认名，部署时通过 Launch 重映射适配。**

关键点

- Launch 文件是管理重映射和命名空间的推荐方式，优于命令行 `--ros-args`
- Python: `remappings=[("/chatter", "/my_topic")]` , `namespace='robot1'`
- XML: `<remap from="..." to="..." />` , `namespace="..."`
- 命名空间为节点所有接口（话题、服务、参数）添加统一前缀
- 多机器人系统通过命名空间隔离，通过重映射对接不同命名的接口

讲次 5：什么是 ROS 2 Parameter

知识点

在前几节中，节点的行为（发布频率、话题名、阈值等）大多硬编码在源码中。调整时至少要修改源码并重启，C++ 还要重新编译，这在开发和部署中很不方便。

ROS 2 Parameter（参数） 提供了一种在**运行时**配置节点行为的机制，无需修改源码或重新编译。

Parameter 的核心概念

概念	说明
键值对	每个参数有名称 (key) 和值 (value) ，如 <code>frequency: 1.0</code>
类型化	支持 <code>bool</code> 、 <code>int</code> 、 <code>float</code> 、 <code>string</code> 、 <code>bool[]</code> 、 <code>int[]</code> 、 <code>float[]</code> 、 <code>string[]</code> 等类型
节点作用域	参数由节点持有；CLI 分别指定节点全名和参数名，不存在 ROS 1 式中央参数服务器
运行时设置	可请求节点修改参数值；节点可以接受或拒绝，业务行为是否立即变化取决于节点实现
声明机制	默认配置下节点先 <code>declare_parameter</code> 再使用；也可显式启用允许未声明参数等选项

Parameter vs 其他配置方式

方式	修改时机	是否需重编译	适用场景
硬编码常量	编译前	是	永远不变的物理常量
ROS Parameter	运行时	否	可调频率、阈值、开关
YAML 配置文件	启动时	否	集中管理多节点参数
Launch Arguments	启动时	否	选择仿真/实机、选机器人型号

参数名称规则

节点全名: `/<namespace>/<node_name>`
参数名: `param_name`（可包含层级分隔点，如 `camera.exposure`）

示例：节点 `/robot1/turtlesim` 持有参数 `background_r`
CLI: `ros2 param get /robot1/turtlesim background_r`

用命令行探索 Turtlesim 参数

```
# 启动 turtlesim
ros2 run turtlesim turtlesim_node
```


新终端中：

```
# 列出某节点的所有参数
ros2 param list /turtlesim
```

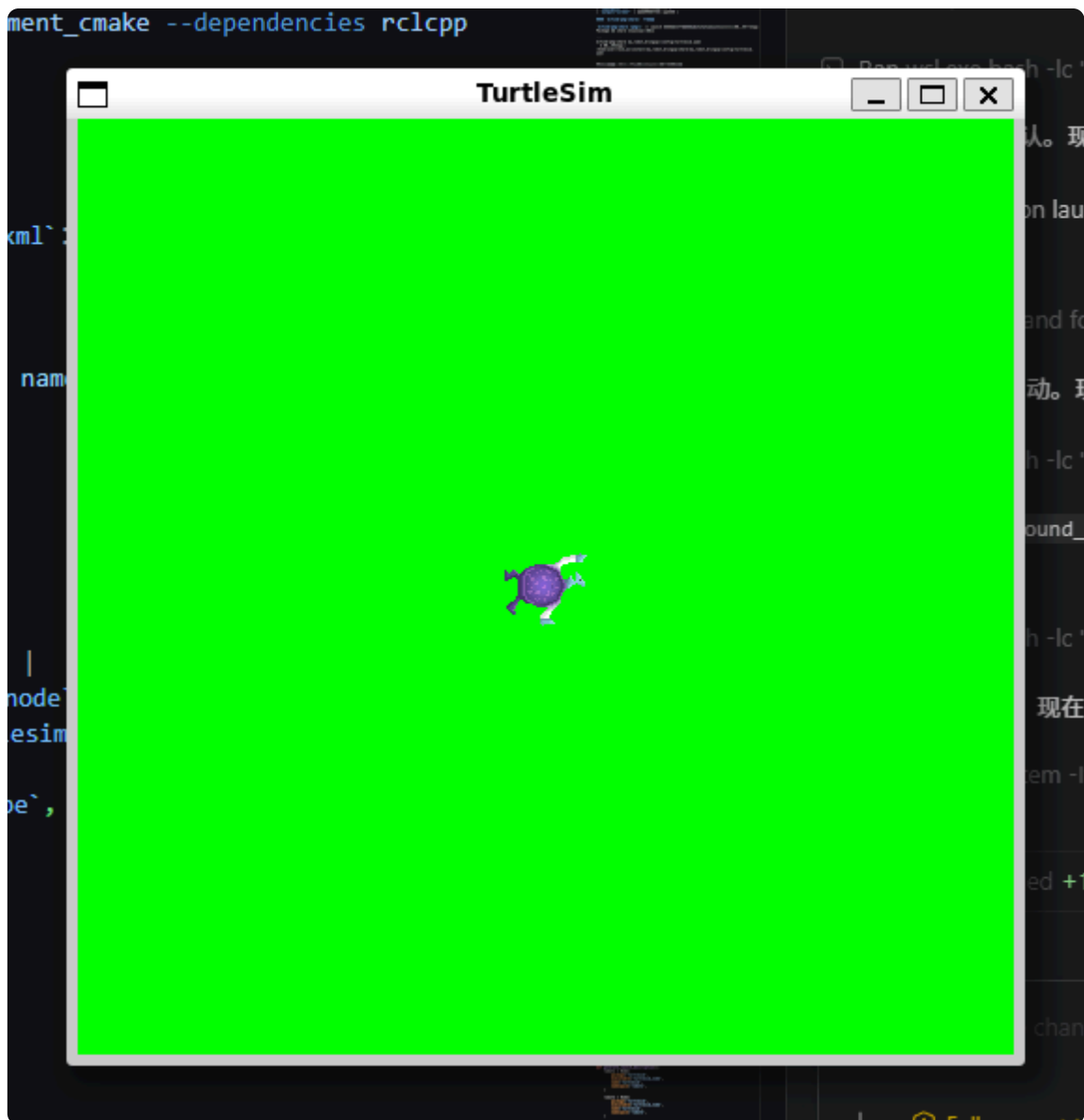
预期输出（部分）：

```
background_b
background_g
background_r
qos_overrides./parameter_events.publisher.depth
qos_overrides./parameter_events.publisher.durability
...
```

```
# 获取参数值
ros2 param get /turtlesim background_r
# 输出：Integer value is: 69

# 请求节点设置参数值：本例 Turtlesim 接受后立即刷新背景
ros2 param set /turtlesim background_r 255
```

Turtlesim 接受该设置并立即刷新背景色。这证明的是 **Turtlesim 实现了动态应用逻辑**；其他节点即使参数值设置成功，也可能只在启动时读取，或通过回调拒绝修改。



上图为本次课件配套命令手动运行 `turtlesim_node` 并通过 `ros2 param set` 修改背景颜色后截取的真实实验窗口画面。

参数的生命周期

启动时

1. `declare_parameter()`
声明参数 + 默认值
2. Launch 文件 / YAML 覆盖
初始值
3. `get_parameter()` 读取
节点使用参数值

运行时

4. `ros2 param set` (外部修改)
或 `add_on_set_parameters_callback`
(回调响应变更)

参数来源优先级

当同一参数有多个来源时，优先级从低到高：

1. 代码中 `declare_parameter()` 的默认值 ← 最低
2. YAML 参数文件（通过 `--params-file` 或 `Launch parameters=[path]`）
3. 命令行 `--ros-args -p name:=value` ← 最高

说明： Launch 文件本身不是独立的优先级层——它内部通过 `--params-file`（加载 YAML）或 `-p`（传入单个参数）来传递参数给节点。如果 Launch 文件同时传入 YAML 文件和 `{'key': value}` 字典，字典（等效于 `-p`）优先于 YAML 文件。

参数事件

ROS 2 提供 `/parameter_events` 话题，当任何参数被修改时会发布事件。节点可通过回调函数监听并响应自己参数的变化（讲次 11 详述）。

关键点

- Parameter 是 ROS 2 的运行时配置机制，键值对、类型化、可按节点组织
 - CLI 以“节点全名 + 参数名”定位参数，例如 `/robot1/turtlesim` 与 `background_r`
 - 默认配置下先 `declare_parameter` 再读取；是否允许未声明参数取决于 `NodeOptions`
 - `ros2 param list/get/set` 是调试参数的核心命令行工具
 - YAML 文件 + Launch 文件是管理多节点参数的推荐方式
-

讲次 6: 【PY】在节点中使用 Parameter

知识点

本节先实现参数声明、启动时覆盖和运行时设置请求。节点如何校验参数并让频率、定时器等行为动态变化，将在讲次 11 加入回调；沿用 2.1 的 OOP 节点风格。

创建带参数的 Publisher 节点

在 `my_py_pkg` 中创建 `my_py_pkg/param_publisher.py`：

```
#!/usr/bin/env python3
import rclpy
from rclpy.executors import ExternalShutdownException
from rclpy.node import Node
from std_msgs.msg import String

class ParamPublisherNode(Node):
    def __init__(self):
        super().__init__('param_publisher')

        # 声明参数（名称，默认值）
        self.declare_parameter('frequency', 1.0)
        self.declare_parameter('message', 'Hello ROS 2')

        # 读取参数值
        frequency = self.get_parameter('frequency').value
        self.message_ = self.get_parameter('message').value
        if frequency <= 0.0:
            raise ValueError('frequency must be positive')

        self.publisher_ = self.create_publisher(String, 'chatter', 10)
        self.timer_ = self.create_timer(1.0 / frequency, self.timer_callback)

        self.get_logger().info(
            f'Publishing at {frequency} Hz: "{self.message_}"'
        )

    def timer_callback(self):
        msg = String()
        msg.data = self.message_
        self.publisher_.publish(msg)

def main(args=None):
    rclpy.init(args=args)
    node = ParamPublisherNode()
    try:
        rclpy.spin(node)
    except (KeyboardInterrupt, ExternalShutdownException):
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()
```

注册入口点

在 `setup.py` 的 `entry_points` 中添加：

```
'param_publisher = my_py_pkg.param_publisher:main',
```

构建与运行

```
colcon build --packages-select my_py_pkg
source install/setup.bash
ros2 run my_py_pkg param_publisher
```

运行时修改参数

```
# 查看当前参数
ros2 param list /param_publisher
ros2 param get /param_publisher frequency

# 修改发布频率（需配合回调才能动态生效，见讲次 11）
ros2 param set /param_publisher frequency 2.0
ros2 param set /param_publisher message "Updated message"
```

启动时通过命令行传参

```
ros2 run my_py_pkg param_publisher --ros-args \
  -p frequency:=0.5 \
  -p message:="Custom hello"
```

核心 API 速查

操作	Python API
声明参数	<code>self.declare_parameter('name', default_value)</code>
读取参数	<code>self.get_parameter('name').value</code>
声明但不设默认值	<code>self.declare_parameter('name')</code>
批量声明	<code>self.declare_parameters('prefix', [('a', 1), ('b', 2)])</code>
检查是否已声明	<code>self.has_parameter('name')</code>
删除参数	<code>self.undeclare_parameter('name')</code>

参数类型与 Python 类型对应

ROS 类型	Python 类型	示例默认值
bool	bool	True
int	int	42
float	float	1.0
string	str	'hello'
数组	list	[1, 2, 3]

在 Launch 文件中为 Python 节点传参

```
Node(  
    package='my_py_pkg',  
    executable='param_publisher',  
    name='param_publisher',  
    parameters=[  
        {'frequency': 2.0},  
        {'message': 'Launch configured'},  
    ],  
)
```

关键点

- `declare_parameter('name', default)` 声明参数, `get_parameter('name').value` 读取
 - 参数可在运行时通过 `ros2 param set` 修改, 也可在启动时通过 `--ros-args -p` 传入
 - Launch 文件的 `parameters=[{...}]` 可在启动时覆盖默认值
 - 若要在运行时动态响应参数变化, 需注册 Parameter Callback (讲次 11)
 - 始终为参数提供合理的默认值, 确保无外部配置时节点也能正常工作
-

讲次 7: 【C++】在节点中使用 Parameter

知识点

C++ 节点参数 API 与 Python 对称，但类型获取使用 `.as_<type>()` 方法而非 `.value` 属性。

创建带参数的 Publisher 节点

在 `my_cpp_pkg/src/param_publisher.cpp` :


```

#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/string.hpp"

using namespace std::chrono_literals;

class ParamPublisherNode : public rclcpp::Node
{
public:
    ParamPublisherNode() : Node("param_publisher")
    {
        // 声明参数
        this->declare_parameter("frequency", 1.0);
        this->declare_parameter("message", "Hello ROS 2");

        // 读取参数
        double frequency = this->get_parameter("frequency").as_double();
        message_ = this->get_parameter("message").as_string();

        publisher_ = this->create_publisher<std_msgs::msg::String>("chatter", 10);
        timer_ = this->create_wall_timer(
            std::chrono::duration<double>(1.0 / frequency),
            std::bind(&ParamPublisherNode::timerCallback, this));

        RCLCPP_INFO(this->get_logger(),
            "Publishing at %.1f Hz: \"%s\"", frequency, message_.c_str());
    }

private:
    void timerCallback()
    {
        auto msg = std_msgs::msg::String();
        msg.data = message_;
        publisher_->publish(msg);
    }

    std::string message_;
    rclcpp::Publisher<std_msgs::msg::String>::SharedPtr publisher_;
    rclcpp::TimerBase::SharedPtr timer_;
};

int main(int argc, char **argv)
{
    rclcpp::init(argc, argv);
    auto node = std::make_shared<ParamPublisherNode>();
    rclcpp::spin(node);
    rclcpp::shutdown();
    return 0;
}

```

注册可执行文件

CMakeLists.txt :

```
add_executable(param_publisher src/param_publisher.cpp)
ament_target_dependencies(param_publisher rclcpp std_msgs)
install(TARGETS param_publisher DESTINATION lib/${PROJECT_NAME})
```

构建与运行

```
colcon build --packages-select my_cpp_pkg
source install/setup.bash
ros2 run my_cpp_pkg param_publisher
```

运行时修改参数

```
ros2 param get /param_publisher frequency
# 当前讲次的简化代码只在启动时创建一次定时器：set 成功后频率不会改变，讲次 11 再补动态回调
ros2 param set /param_publisher frequency 2.0
ros2 param set /param_publisher message "Updated from CLI"
```

启动时通过命令行传参

```
ros2 run my_cpp_pkg param_publisher --ros-args \
  -p frequency:=0.5 \
  -p message:="Custom hello"
```

核心 API 速查

操作	C++ API
声明参数	<code>this->declare_parameter("name", default_value)</code>
读取 int	<code>this->get_parameter("name").as_int()</code>
读取 double	<code>this->get_parameter("name").as_double()</code>
读取 string	<code>this->get_parameter("name").as_string()</code>
读取 bool	<code>this->get_parameter("name").as_bool()</code>
读取数组	<code>this->get_parameter("name").as_integer_array()</code>
批量声明	<code>this->declare_parameters("prefix", {{"a", 1}, {"b", 2}})</code>

Python vs C++ 参数 API 对照

操作	Python	C++
声明	<code>declare_parameter('freq', 1.0)</code>	<code>declare_parameter("freq", 1.0)</code>
读取 float	<code>.get_parameter('freq').value</code>	<code>.get_parameter("freq").as_double()</code>
读取 string	<code>.get_parameter('msg').value</code>	<code>.get_parameter("msg").as_string()</code>
读取 int	<code>.get_parameter('num').value</code>	<code>.get_parameter("num").as_int()</code>

在 Launch 文件中为 C++ 节点传参

```
Node(  
    package='my_cpp_pkg',  
    executable='param_publisher',  
    name='param_publisher',  
    parameters=[  
        {'frequency': 2.0},  
        {'message': 'Launch configured'}  
    ],  
)
```

关键点

- C++ 使用 `declare_parameter("name", default)` 声明, `get_parameter("name").as_<type>()` 读取
 - `.as_double()`、`.as_string()`、`.as_int()` 等提供类型安全的访问
 - 命令行和 Launch 文件的传参方式与 Python 节点完全一致
 - 参数系统是 ROS 2 的跨语言特性, Python 和 C++ 节点参数可通过同一套 CLI 工具管理
-

讲次 8: 【LAB】Turtlesim Parameter 实验

实验目标

通过操作 Turtlesim 内置参数，掌握 `ros2 param` 命令行工具的完整用法，直观感受参数动态更新的效果。

实验步骤

步骤 1: 启动 Turtlesim

```
source /opt/ros/jazzy/setup.bash
ros2 run turtlesim turtlesim_node
```

步骤 2: 列出所有参数

新开终端：

```
source /opt/ros/jazzy/setup.bash
ros2 param list /turtlesim
```

观察输出中的颜色参数：

```
background_b
background_g
background_r
```

以及 QoS 相关参数（系统自动生成，可忽略）。

步骤 3: 获取参数当前值

```
ros2 param get /turtlesim background_r
ros2 param get /turtlesim background_g
ros2 param get /turtlesim background_b
```

预期输出示例：

```
Integer value is: 69
Integer value is: 204
Integer value is: 239
```

三个值共同决定窗口背景色（默认浅蓝色）。

步骤 4: 修改背景色

```
# 将红色分量设为 255，背景立即变红
ros2 param set /turtlesim background_r 255

# 设为绿色背景
ros2 param set /turtlesim background_r 0
ros2 param set /turtlesim background_g 255
ros2 param set /turtlesim background_b 0

# 恢复默认浅蓝色
ros2 param set /turtlesim background_r 69
ros2 param set /turtlesim background_g 204
ros2 param set /turtlesim background_b 239
```

每次 `set` 后观察 Turtlesim 窗口——颜色应立即变化，无需重启节点。

步骤 5: 批量导出与加载参数

```
# 导出当前所有参数到文件
ros2 param dump /turtlesim > /tmp/turtlesim_params.yaml

# 查看导出的 YAML 格式
cat /tmp/turtlesim_params.yaml
```

输出格式:

```
/turtlesim:
  ros__parameters:
    background_b: 239
    background_g: 204
    background_r: 69
    qos_overrides:
      ...
```

```
# 从文件加载参数（覆盖当前值）
ros2 param load /turtlesim /tmp/turtlesim_params.yaml
```

步骤 6: 查看参数描述

```
ros2 param describe /turtlesim background_r
```

实验检查清单

- [] 能使用 `ros2 param list` 列出节点所有参数
- [] 能使用 `ros2 param get` 查看参数值
- [] 能使用 `ros2 param set /turtlesim background_r 255` 动态修改背景色

- [] 理解参数值能否修改、行为是否立即变化都由节点实现决定；本例 Turtlesim 会立即刷新背景
- [] 能使用 `ros2 param dump/load` 导出和导入参数
- [] 理解 YAML 参数文件的基本格式（`ros__parameters` 键）

关键点

- `ros2 param list/get/set` 是调试参数的三大核心命令
 - `set` 成功只表示节点接受了新值；业务逻辑是否立即使用新值，需要看参数回调或读取方式
 - `ros2 param dump` 导出的 YAML 格式可直接用于 Launch 文件加载
 - Turtlesim 的 `background_r/g/b` 是体验参数动态更新的最佳入口
-

讲次 9: Launch 文件与 Parameters 集成

知识点

单独用 `ros2 param set` 调试后，我们需要把参数配置固化到 Launch 文件中，实现"一键启动 + 自动配置"。

方式一：Launch 文件内联参数

XML:

```
<launch>
  <node pkg="turtlesim" exec="turtlesim_node" name="turtlesim">
    <param name="background_r" value="255"/>
    <param name="background_g" value="0"/>
    <param name="background_b" value="0"/>
  </node>
</launch>
```

Python:

```
Node(
  package='turtlesim',
  executable='turtlesim_node',
  name='turtlesim',
  parameters=[
    {'background_r': 255},
    {'background_g': 0},
    {'background_b': 0},
  ],
)
```

启动后 Turtlesim 窗口直接显示红色背景。

方式二：通过 YAML 文件加载（推荐）

当参数数量较多或需要跨节点共享时，使用 YAML 文件集中管理（讲次 10 详述格式）。

XML:

```
<launch>
  <node pkg="turtlesim" exec="turtlesim_node" name="turtlesim">
    <param from="$(find-pkg-share my_robot_bringup)/config/turtlesim.yaml"/>
  </node>
</launch>
```

Python:

```

from ament_index_python.packages import get_package_share_directory
import os

def generate_launch_description():
    config_file = os.path.join(
        get_package_share_directory('my_robot_bringup'),
        'config', 'turtlesim.yaml'
    )

    return LaunchDescription([
        Node(
            package='turtlesim',
            executable='turtlesim_node',
            name='turtlesim',
            parameters=[config_file], # 直接传入 YAML 文件路径
        ),
    ])

```

方式三：Launch Arguments 传递参数

```

from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    bg_r_arg = DeclareLaunchArgument('bg_r', default_value='69')

    return LaunchDescription([
        bg_r_arg,
        Node(
            package='turtlesim',
            executable='turtlesim_node',
            name='turtlesim',
            parameters=[{
                'background_r': LaunchConfiguration('bg_r'),
            }],
        ),
    ])

```

```

# 使用默认值
ros2 launch my_robot_bringup turtlesim.launch.py

# 命令行覆盖
ros2 launch my_robot_bringup turtlesim.launch.py bg_r:=255

```

为自定义节点传参的完整 Launch 示例

number_app.launch.py (仓库 `stage2_code/my_robot_bringup/launch/number_app.launch.py` , 同一份 YAML 同时配置 Python 和 C++ 两个节点) :


```
import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    pkg_share = get_package_share_directory("my_robot_bringup")
    config_file = os.path.join(pkg_share, "config", "number_app.yaml")

    python_publisher = Node(
        package="my_py_pkg",
        executable="param_publisher",
        name="param_publisher",
        parameters=[config_file],
        output="screen",
    )

    cpp_publisher = Node(
        package="my_cpp_pkg",
        executable="param_publisher",
        name="cpp_param_publisher",
        parameters=[config_file],
        output="screen",
    )

    return LaunchDescription([python_publisher, cpp_publisher])
```

配套的 `config/number_app.yaml` (顶层键与 Launch 中的 `name` 一一对应) :

```
/param_publisher:
  ros__parameters:
    frequency: 1.5
    message: "Python YAML configured"

/cpp_param_publisher:
  ros__parameters:
    frequency: 1.0
    message: "C++ YAML configured"
```

安装 config 目录

`CMakeLists.txt` 中还需安装 config 目录:

```
install(DIRECTORY launch config
  DESTINATION share/${PROJECT_NAME}
)
```

关键点

- Launch 文件传参三种方式: 内联字典、`parameters=[yaml_file]`、Launch Arguments

- `parameters=[yaml_file]` 是管理多参数的首选方式
 - Python 用 `parameters=[config_file]` , XML 用 `<param from="..."/>`
 - 自定义节点和官方节点的传参方式完全一致
 - 记得在 `CMakeLists.txt` 中 `install(DIRECTORY config ...)` 安装配置文件
-

讲次 10: YAML 参数文件

知识点

YAML 参数文件是 ROS 2 项目中管理参数的标准方式。它将参数配置从代码和 Launch 文件中分离出来，便于非开发人员（如产品经理、测试工程师）调整系统行为。

基本格式

```
/**:
  ros__parameters:
    param1: value1
    param2: value2
```

关键规则：

1. 顶层键是**节点全名**（含前导 `/`），`/**` 表示通配所有节点
2. 必须有 `ros__parameters` 键（注意双下划线）
3. 缩进使用空格（不用 Tab）

单节点参数文件

`my_robot_bringup/config/turtlesim.yaml`：

```
/turtlesim:
  ros__parameters:
    background_r: 69
    background_g: 204
    background_b: 239
```

多节点参数文件

一个 YAML 文件可为多个节点配置参数：

```
/number_publisher:
  ros__parameters:
    number: 6
    timer_period: 1.5

/number_counter:
  ros__parameters:
    threshold: 100
```

带命名空间的节点

```
/demo/number_publisher:  
  ros__parameters:  
    number: 6  
    timer_period: 1.5
```

节点全名必须与 Launch 文件中的 `namespace` + `name` 一致。

参数类型示例

```
/my_node:  
  ros__parameters:  
    # 标量类型  
    enable_feature: true      # bool  
    max_count: 42             # int  
    frequency: 1.5            # float  
    robot_name: "robot_01"    # string  
  
    # 数组类型  
    joint_names: ["joint1", "joint2", "joint3"]  
    pid_gains: [1.0, 0.5, 0.1]  
    sensor_ids: [1, 2, 3, 4]
```

在 Launch 中加载 YAML

Python (推荐写法) :

```
parameters=[config_file] # config_file 是 YAML 文件的绝对路径
```

XML:

```
<param from="$(find-pkg-share my_robot_bringup)/config/turtlesim.yaml"/>
```

用命令行工具验证 YAML

```
# 先启动节点  
ros2 run turtlesim turtlesim_node  
  
# 从另一个终端加载 YAML  
ros2 param load /turtlesim \  
  ~/ros2_ws/install/my_robot_bringup/share/my_robot_bringup/config/turtlesim.yaml  
  
# 验证  
ros2 param get /turtlesim background_r  
# 输出: Integer value is: 255
```

从运行中的节点导出 YAML

```
ros2 param dump /turtlesim > turtlesim_backup.yaml
```

导出的文件可直接作为配置文件使用或纳入版本管理。

YAML 文件组织最佳实践

```
my_robot_bringup/config/
├─ turtlesim.yaml      ← 仿真相关参数
├─ navigation.yaml     ← 导航节点参数
├─ perception.yaml     ← 感知节点参数
└─ default.yaml        ← 全局默认参数
```

每个 YAML 对应一个功能模块，Launch 文件按需加载：

```
Node(
  package='nav2_planner',
  executable='planner_server',
  parameters=[
    os.path.join(config_dir, 'navigation.yaml'),
    os.path.join(config_dir, 'default.yaml'), # 可叠加多个文件
  ],
)
```

关键点

- YAML 参数文件必须有 `ros__parameters` 键，节点名作为顶层键
 - 一个 YAML 文件可配置多个节点，节点全名必须与 Launch 中 namespace + name 一致
 - `parameters=[yaml_file]` 是 Launch 加载 YAML 的标准写法
 - `ros2 param dump/load` 可在运行时导出和导入 YAML 配置
 - 按功能模块组织 YAML 文件，便于团队协作和版本管理
-

讲次 11：高级 Parameter Callbacks

知识点

前面讲次中，运行时 `ros2 param set` 修改参数后，节点并不会自动响应——因为 `get_parameter` 只在初始化时调用了一次。要让节点**动态响应参数变化**，需要注册 Parameter Callback。

ROS 2 Jazzy 提供三个参数回调阶段；本节重点使用 On Set 做校验、Post Set 应用副作用：

回调类型	注册方法	触发时机	能否拒绝修改
Pre Set	<code>add_pre_set_parameters_callback</code>	校验前，可补充或转换本次参数列表	通过返回列表间接控制
On Set	<code>add_on_set_parameters_callback</code>	参数写入 之前	能
Post Set	<code>add_post_set_parameters_callback</code>	参数写入 之后	不能

【PY】 On Set Callback (可拒绝非法值)

```
from rclpy.node import Node
from rcl_interfaces.msg import SetParametersResult
from rclpy.parameter import Parameter

class ParamPublisherNode(Node):
    def __init__(self):
        super().__init__('param_publisher')
        self.declare_parameter('frequency', 1.0)
        self.frequency_ = self.get_parameter('frequency').value

        self.timer_ = self.create_timer(
            1.0 / self.frequency_, self.timer_callback)

        self.on_set_handle_ = self.add_on_set_parameters_callback(
            self.on_set_parameters)
        self.post_set_handle_ = self.add_post_set_parameters_callback(
            self.post_set_parameters)

    def timer_callback(self):
        self.get_logger().info(f'Publishing at {self.frequency_: .1f} Hz')

    def on_set_parameters(self, params: list[Parameter]):
        for param in params:
            if param.name == 'frequency':
                if param.value <= 0.0:
                    return SetParametersResult(
                        successful=False,
                        reason='frequency must be positive')
        return SetParametersResult(successful=True)

    def post_set_parameters(self, params: list[Parameter]):
        for param in params:
            if param.name == 'frequency':
                self.frequency_ = float(param.value)
                self.timer_.cancel()
                self.timer_ = self.create_timer(
                    1.0 / self.frequency_, self.timer_callback)
```

On Set 可能与其他校验回调共同执行，因此只做快速、无副作用的合法性检查。Timer 重建等行为放在 Post Set，确保参数已经成功写入。

【PY】 Post Set Callback (同步内部状态)

```
# 在上一个 post_set_parameters() 的同一 for 循环中追加
elif param.name == 'message':
    self.message_ = str(param.value)
    self.get_logger().info(f'Message updated to: {self.message_}')
```

[C++] On Set Callback

```
#include "rcl_interfaces/msg/set_parameters_result.hpp"

class ParamPublisherNode : public rclcpp::Node
{
public:
    ParamPublisherNode() : Node("param_publisher")
    {
        this->declare_parameter("frequency", 1.0);
        frequency_ = this->get_parameter("frequency").as_double();

        on_set_handle_ = this->add_on_set_parameters_callback(
            std::bind(&ParamPublisherNode::onSetParameters, this, std::placeholders::_1));
        post_set_handle_ = this->add_post_set_parameters_callback(
            std::bind(&ParamPublisherNode::postSetParameters, this, std::placeholders::_1));

        timer_ = this->create_wall_timer(
            std::chrono::duration<double>(1.0 / frequency_),
            std::bind(&ParamPublisherNode::timerCallback, this));
    }

private:
    rcl_interfaces::msg::SetParametersResult onSetParameters(
        const std::vector<rclcpp::Parameter> & params)
    {
        rcl_interfaces::msg::SetParametersResult result;
        result.successful = true;

        for (const auto & param : params) {
            if (param.get_name() == "frequency") {
                if (param.as_double() <= 0.0) {
                    result.successful = false;
                    result.reason = "frequency must be positive";
                    return result;
                }
            }
        }
        return result;
    }

    void postSetParameters(const std::vector<rclcpp::Parameter> & params)
    {
        for (const auto & param : params) {
            if (param.get_name() == "frequency") {
                frequency_ = param.as_double();
                timer_->cancel();
                timer_ = this->create_wall_timer(
                    std::chrono::duration<double>(1.0 / frequency_),
                    std::bind(&ParamPublisherNode::timerCallback, this));
            }
        }
    }

    double frequency_;
    rclcpp::TimerBase::SharedPtr timer_;
```



```
rclcpp::node_interfaces::OnSetParametersCallbackHandle::SharedPtr on_set_handle_;
rclcpp::node_interfaces::PostSetParametersCallbackHandle::SharedPtr post_set_handle_;
};
```

测试回调

```
ros2 run my_py_pkg param_publisher

# 合法修改 → 成功
ros2 param set /param_publisher frequency 2.0

# 非法修改 → 被拒绝
ros2 param set /param_publisher frequency -1.0
# 输出: Set parameter failed: frequency must be positive
```

回调选型指南

需要在写入前校验参数合法性（范围检查、类型约束）？

- └ 是 → `add_on_set_parameters_callback`
- └ 否 → 只是写入后刷新内部状态？
 - └ 是 → `add_post_set_parameters_callback`（更简单）
 - └ 否 → 无需回调，启动时读取一次即可

注意事项

- 回调中**避免耗时操作**，否则会阻塞参数服务
- On Set 回调中如果返回 `successful=False`，参数值不会被修改
- On Set 只做校验；Timer 重建、缓存刷新等副作用放在 Post Set，避免事务最终被其他回调拒绝后内部状态已经改变
- 必须为回调返回值保存句柄（C++），否则回调会被自动注销：

```
// C++ 中必须保存句柄
rclcpp::node_interfaces::OnSetParametersCallbackHandle::SharedPtr on_set_handle_;
rclcpp::node_interfaces::PostSetParametersCallbackHandle::SharedPtr post_set_handle_;
```

关键点

- `add_on_set_parameters_callback` 在参数写入前触发，可校验并拒绝非法值
 - `add_post_set_parameters_callback` 在参数写入后触发，用于刷新内部状态
 - Python 回调返回 `SetParametersResult(successful=True/False)`
 - C++ 回调返回 `rcl_interfaces::msg::SetParametersResult`，必须保存回调句柄
 - 回调是实现"运行时动态重配置"的关键机制
-

讲次 12：【LAB】启动你的第一个 ROS 2 程序

实验目标

综合运用本节所学，创建一个完整的 Launch 文件，一键启动 **turtlesim 仿真器 + 键盘控制 + 自定义节点**，并配置参数、重映射和命名空间。

实验准备

确保已有以下 Package（参考 2.1 ~ 2.6 创建）：

- `my_robot_bringup`（ament_cmake，存放 Launch 和配置）
- `my_py_pkg`（含 `param_publisher` 或自定义节点）

实验步骤

步骤 1：创建 YAML 参数文件

`my_robot_bringup/config/bringup.yaml`：

```
/turtlesim:
  ros__parameters:
    background_r: 30
    background_g: 100
    background_b: 200

/param_publisher:
  ros__parameters:
    frequency: 1.0
    message: "Turtlesim is running!"
```

步骤 2：创建自定义节点（如尚未创建）

`my_py_pkg/my_py_pkg/param_publisher.py`（参见讲次 6），确保已注册入口点并编译通过。

步骤 3：编写综合 Launch 文件

`my_robot_bringup/launch/bringup.launch.py`：

```

import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.conditions import IfCondition
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():
    pkg_share = get_package_share_directory('my_robot_bringup')
    config_file = os.path.join(pkg_share, 'config', 'bringup.yaml')
    with_teleop = LaunchConfiguration('with_teleop')

    turtlesim_node = Node(
        package='turtlesim',
        executable='turtlesim_node',
        name='turtlesim',
        output='screen',
        parameters=[config_file],
    )

    teleop_node = Node(
        package='turtlesim',
        executable='turtle_teleop_key',
        name='teleop',
        output='screen',
        emulate_tty=True,
        prefix='bash -lc \'exec </dev/tty; exec "$@"\' bash',
        condition=IfCondition(with_teleop),
    )

    param_publisher_node = Node(
        package='my_py_pkg',
        executable='param_publisher',
        name='param_publisher',
        output='screen',
        remappings=[('/chatter', '/my_topic')],
        parameters=[config_file],
    )

    return LaunchDescription([
        DeclareLaunchArgument(
            'with_teleop',
            default_value='true',
            description='Start the interactive keyboard teleop node',
        ),
        turtlesim_node,
        teleop_node,
        param_publisher_node,
    ])

```

步骤 4: 配置安装规则

my_robot_bringup/CMakeLists.txt :

```
install(DIRECTORY launch config
  DESTINATION share/${PROJECT_NAME}
)
```

步骤 5: 构建

```
cd ~/ros2_ws
colcon build --packages-select my_robot_bringup my_py_pkg
source install/setup.bash
```

步骤 6: 一键启动

```
ros2 launch my_robot_bringup bringup.launch.py

# CI、录屏脚本或无交互终端中关闭键盘节点
ros2 launch my_robot_bringup bringup.launch.py with_teleop:=false
```

步骤 7: 验证系统

新开终端，逐项检查：

```
source ~/ros2_ws/install/setup.bash

# 1. 检查所有节点已启动
ros2 node list
# 预期:
# /param_publisher
# /teleop
# /turtlesim

# 2. 检查参数已加载
ros2 param get /turtlesim background_r
# 预期: Integer value is: 30

ros2 param get /param_publisher frequency
# 预期: Double value is: 1.0

# 3. 检查自定义话题（注意重映射）
ros2 topic echo /my_topic

# 4. 动态修改参数
ros2 param set /turtlesim background_r 255
ros2 param set /param_publisher frequency 2.0
ros2 param set /param_publisher message "Runtime update"

# 5. 用键盘控制海龟（在 teleop 终端中操作）
# 方向键控制海龟移动
```

步骤 8: 优雅关闭

在 Launch 终端按 `Ctrl+C`，确认所有节点同时关闭：

```
ros2 node list
# 预期：无输出（所有节点已关闭）
```

扩展练习（选做）

1. 为 `bringup.launch.py` 添加 Launch Argument `bg_r`，支持命令行自定义背景色
2. 在 `param_publisher` 中添加 `add_on_set_parameters_callback`，运行时修改 `frequency` 立即改变发布频率
3. 添加命名空间 `namespace='lab'`，观察所有接口名称的变化
4. 编写等价的 XML Launch 文件 `bringup.launch.xml`，对比两种写法

实验检查清单

- ☐ Launch 文件成功启动 3 个节点 (turtlesim + teleop + param_publisher)
- ☐ YAML 参数文件正确加载（背景色、发布频率）
- ☐ 重映射生效（`/chatter` → `/my_topic`）
- ☐ `ros2 param set` 可动态修改参数
- ☐ `Ctrl+C` 优雅关闭所有节点

- [] 理解 Launch + YAML + 重映射 + 参数的完整集成流程

关键点

- 综合 Launch 文件是 ROS 2 项目的"启动入口", 集成节点、参数、重映射于一体
 - `bringup` Package 是社区通用的命名惯例, 专门存放 Launch 和配置文件
 - 一条 `ros2 launch` 命令替代多个 `ros2 run`, 是正式项目的标准做法
 - 本实验涵盖了 2.6 全部知识点, 是进入 2.7 实战项目的必要准备
-

本节总结

命令速查表

命令	说明
<code>ros2 launch <pkg> <file></code>	执行 Launch 文件，启动一个或多个节点
<code>ros2 launch <pkg> <file> arg:=value</code>	传入 Launch Argument 覆盖默认值
<code>ros2 param list <node></code>	列出指定节点的所有参数
<code>ros2 param list</code>	列出系统中所有参数
<code>ros2 param get <node> <param></code>	获取参数值
<code>ros2 param set <node> <param> <value></code>	请求节点设置参数；是否接受及行为何时变化由节点实现
<code>ros2 param set /turtlesim background_r 255</code>	修改 Turtlesim 背景红色分量
<code>ros2 param dump <node> > file.yaml</code>	导出节点参数到 YAML 文件
<code>ros2 param load <node> file.yaml</code>	从 YAML 文件加载参数
<code>ros2 param describe <node> <param></code>	查看参数描述信息
<code>ros2 run <pkg> <node> --ros-args -p name:=value</code>	启动时传入参数
<code>ros2 run <pkg> <node> --ros-args -r __ns:=/robot1</code>	启动时设置命名空间

Launch / Parameter 常用 API 对照

Launch 文件

操作	XML Launch	Python Launch
启动节点	<code><node pkg="..." exec="..." name="..." /></code>	<code>Node(package=..., executable=..., name=...)</code>
话题重映射	<code><remap from="..." to="..." /></code>	<code>remappings=[("/a", "/b")]</code>
命名空间	<code>namespace="robot1"</code>	<code>namespace='robot1'</code>
内联参数	<code><param name="k" value="v" /></code>	<code>parameters=[{'k': v}]</code>
YAML 参数	<code><param from="\$(find-pkg-share pkg)/cfg.yaml" /></code>	<code>parameters=[config_file]</code>
日志到终端	<code>output="screen"</code>	<code>output='screen'</code>
引用资源路径	<code>\$(find-pkg-share pkg)</code>	<code>get_package_share_directory('pkg')</code>
声明启动参数	—	<code>DeclareLaunchArgument('arg', default_value='...')</code>
引用启动参数	—	<code>LaunchConfiguration('arg')</code>

Parameter API

操作	Python (rclpy)	C++ (rclcpp)
声明参数	<code>self.declare_parameter('name', 1.0)</code>	<code>this->declare_parameter("name", 1.0)</code>
读取 float	<code>self.get_parameter('name').value</code>	<code>this->get_parameter("name").as_double()</code>
读取 int	<code>self.get_parameter('name').value</code>	<code>this->get_parameter("name").as_int()</code>
读取 string	<code>self.get_parameter('name').value</code>	<code>this->get_parameter("name").as_string()</code>
读取 bool	<code>self.get_parameter('name').value</code>	<code>this->get_parameter("name").as_bool()</code>
On Set 回调	<code>self.add_on_set_parameters_callback(cb)</code>	<code>this->add_on_set_parameters_callback(cb)</code>
Post Set 回调	<code>self.add_post_set_parameters_callback(cb)</code>	<code>this->add_post_set_parameters_callback(cb)</code>
返回校验结果 (PY)	<code>SetParametersResult(successful=False, reason='...')</code>	—
返回校验结果 (C++)	—	<code>rcl_interfaces::msg::SetParametersResult</code>

YAML 参数文件模板

```
<code>/<node_full_name>:
  ros__parameters:
    param_name: value</code>
```

下一步

下一节 **2.7 Turtlesim 项目** 中，我们将把本节所学的 Launch 文件、参数系统、话题/服务/动作通信全部融会贯通，完成一个完整的 **"Catch Them All"** 海龟抓捕项目。你将编写 Spawner 节点（定时生成海龟）、Controller 节点（驱动海龟抓捕目标），并用 Launch 文件 + YAML 配置一键启动整个系统。这是第二阶段的核心实战，也是进入第三阶段 Gazebo 仿真的跳板。